Compiler Construction Lab - Assignment 9

Name: Dev Vachhani
PRN.: 24070122517
Batch: B3

Objective

To design and implement a parser that checks whether an input string can be generated by a user-defined context-free grammar. The program accepts production rules with alternatives and eps productions, displays the parsing steps in a table, and reports whether the string is accepted or rejected.

Flex tokenizes the grammar notation, and Bison parses the production lines. The input string is parsed by hand-written recursive-descent functions in C.

Theory

A context-free grammar is defined as G = (V, T, P, S), where V is the set of non-terminals, T is the set of terminals, P is the set of production rules, and S is the start symbol. A production has the form A → α, where A is a non-terminal and α is a sequence of terminals and non-terminals. The | symbol separates alternative right-hand sides. The special production eps represents the empty string and consumes no input.

In recursive descent parsing, a function is used to parse each non-terminal. Starting with the grammar's start symbol, the parser tries each applicable production. For a terminal, it compares the terminal with the input at the current index and advances the index on a match. For a non-terminal, it calls the corresponding parsing function. If a choice fails, the parser backtracks and tries another alternative.

The string is accepted if at least one parsing path matches the entire input. A match of only part of the input is not sufficient. Since recursive descent can loop on left-recursive rules, this implementation detects same-position recursive cycles; grammars should avoid left recursion.

Bison normally generates an LALR parser. Here, Bison parses the grammar's production-line notation; the hand-written C functions perform the recursive-descent parsing of the input string.

Screenshots and Execution Results

Problems Output Debug Console Terminal Ports

```
kisna@MacBook-Air-2 Assignment-9 % make
./rdparser
bison -d -o parser.tab.c parser.y
flex -o lexer.yy.c lexer.l
cc -std=c11 -Wall -Wextra -pedantic -O2 -o rdparser parser.c parser.tab.c lexer.yy.c
Recursive Descent Parser
------------------------
Enter grammar productions (one per line; blank line finishes):
S -> a S a | b


Grammar:
S -> a S a | b

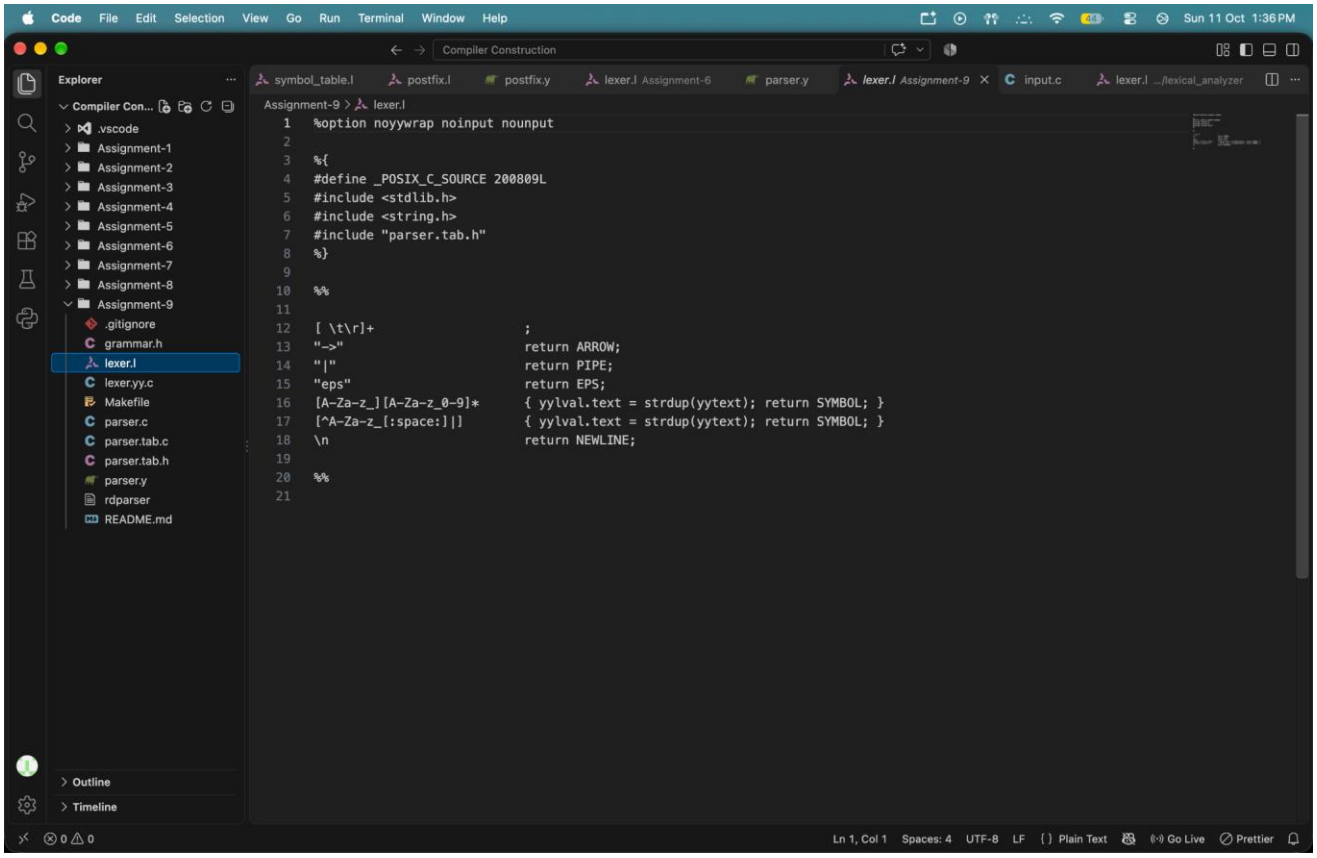
Enter the string to parse: aba

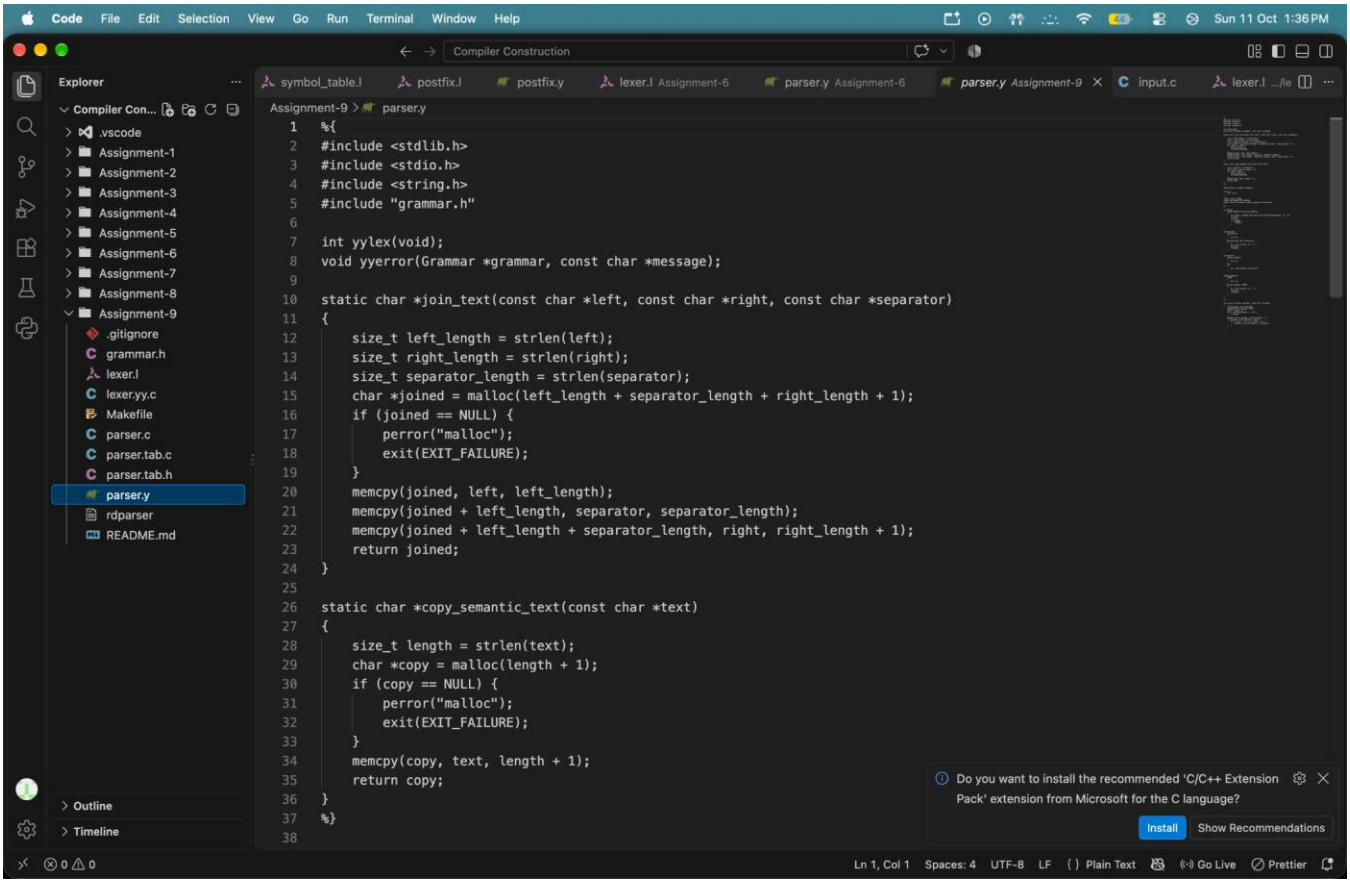
Parsing 'aba' with start symbol 'S'

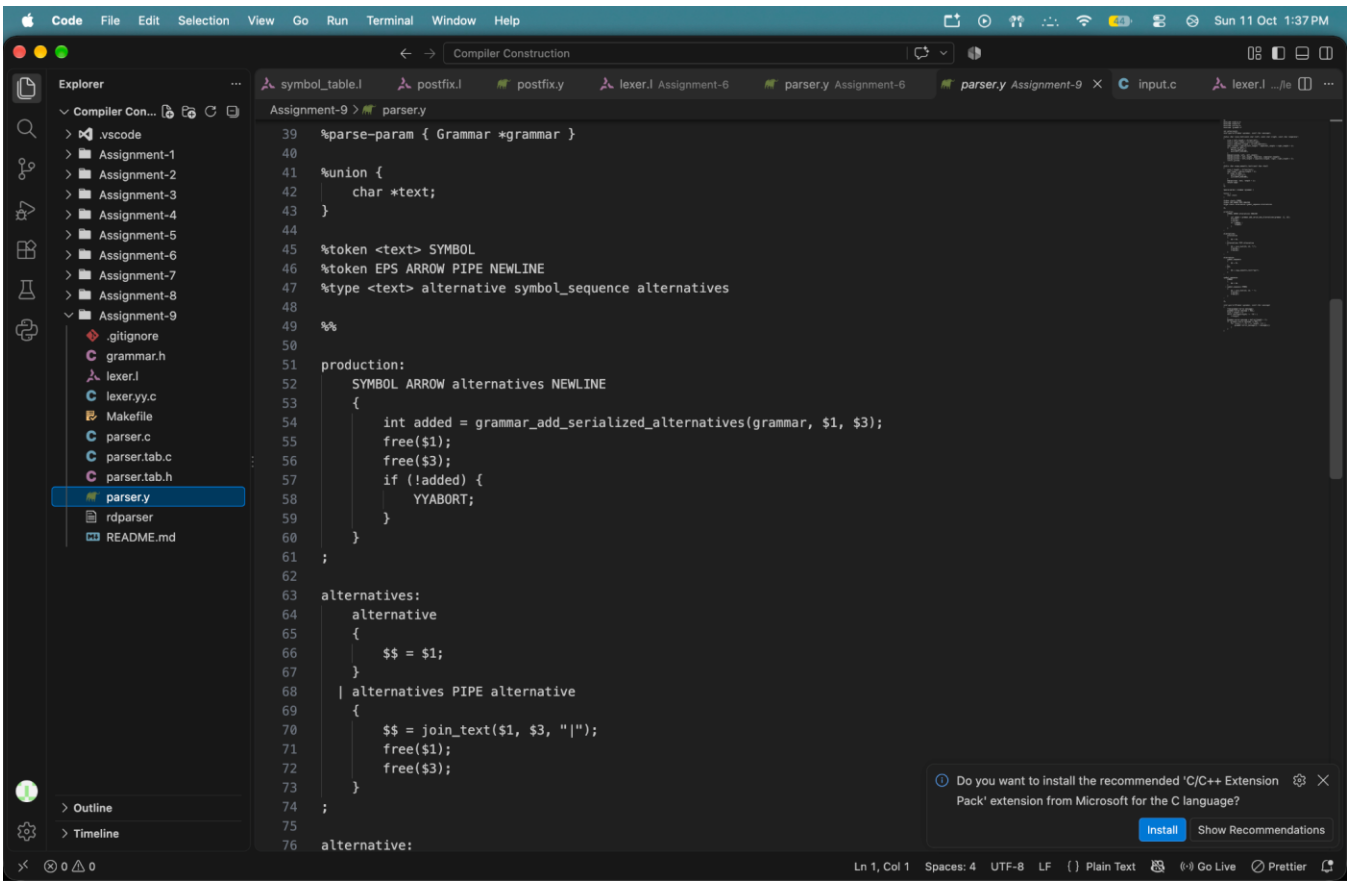
Detailed Parsing Steps:

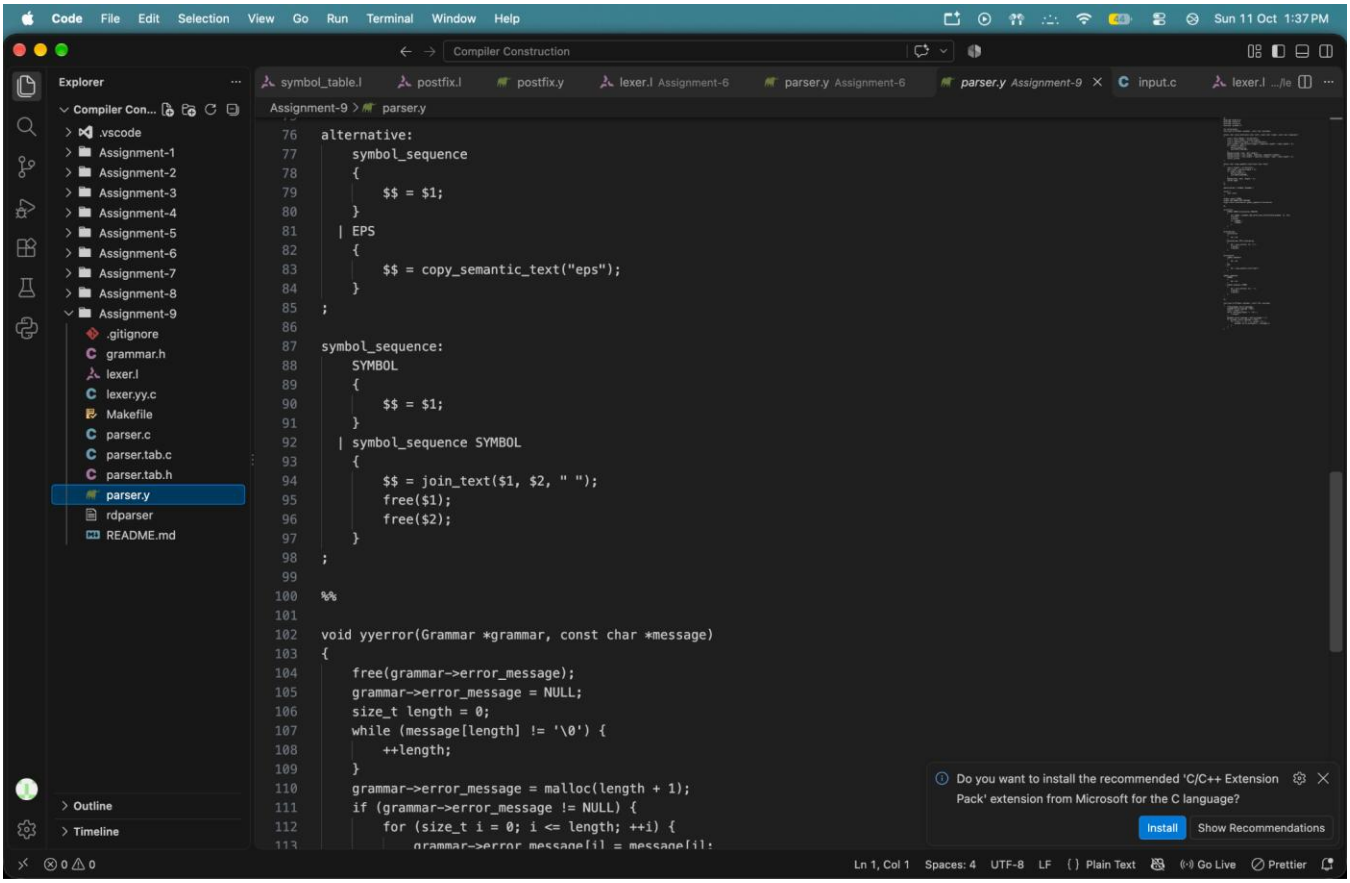
| No. | Type | Entity | Index | Description |
| ---: | --- | --- | ---: | --- |
| 0 | Call | S | 0 | Entering parse_non_terminal('S') at index 0. |
| 1 | Attempt | S | 0 | Trying alternative 1: `a S a` for non-terminal 'S'. |
| 2 | Match | a | 0 | Successfully matched terminal 'a' at input index 0. |
| 3 | Call | S | 1 | Entering parse_non_terminal('S') at index 1. |
| 4 | Attempt | S | 1 | Trying alternative 1: `a S a` for non-terminal 'S'. |
| 5 | Match | a | 1 | Failed to match terminal 'a' at index 1; expected 'a'. |
| 6 | Failure | S | 1 | Alternative 1: `a S a` for 'S' failed. |
| 7 | Attempt | S | 1 | Trying alternative 2: `b` for non-terminal 'S'. |
| 8 | Match | b | 1 | Successfully matched terminal 'b' at input index 1. |
| 9 | Success | S | 2 | Alternative 2: `b` for 'S' succeeded; returning at index 2. |
| 10 | Return | S | 2 | Returning from parse_non_terminal('S') with 1 possible end index(es). |
| 11 | Match | a | 2 | Successfully matched terminal 'a' at input index 2. |
| 12 | Success | S | 3 | Alternative 1: `a S a` for 'S' succeeded; returning at index 3. |
| 13 | Attempt | S | 0 | Trying alternative 2: `b` for non-terminal 'S'. |
| 14 | Match | b | 0 | Failed to match terminal 'b' at index 0; expected 'b'. |
| 15 | Failure | S | 0 | Alternative 2: `b` for 'S' failed. |
| 16 | Return | S | 3 | Returning from parse_non_terminal('S') with 1 possible end index(es). |
| 17 | Result | input | - | Parsing completed successfully. |

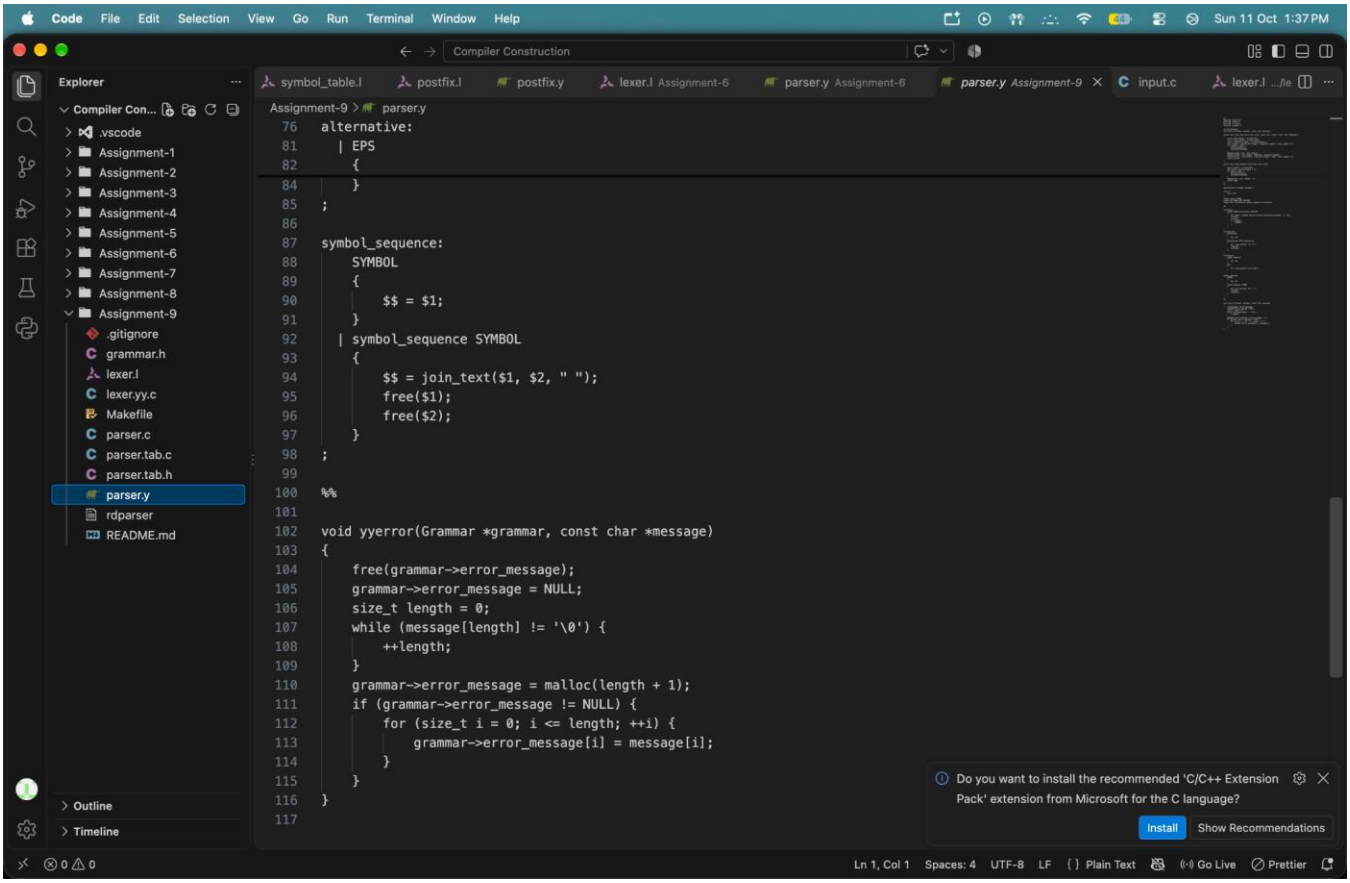
Result: String 'aba' is ACCEPTED by the grammar.
kisna@MacBook-Air-2 Assignment-9 %
```




Code File Edit Selection View Go Run Terminal Window Help
Sun 11 Oct 1:36 PM
Compiler Construction
Explorer
Compiler Con...
.vscode
Assignment-1
Assignment-2
Assignment-3
Assignment-4
Assignment-5
Assignment-6
Assignment-7
Assignment-8
Assignment-9
.gitignore
grammar.h
lexer.l
lexer.yy.c
Makefile
parser.c
parser.tab.c
parser.tab.h
parser.y
rdparser
README.md
symbol_table.l
postfix.l
postfix.y
lexer.l Assignment-6
parser.y Assignment-6
parser.y Assignment-9
input.c
Assignment-9 > parser.y
%{
#include <stdlib.h>
#include <stdio.h>
#include <string.h>
#include "grammar.h"
int yylex(void);
void yyerror(Grammar *grammar, const char *message);
static char *join_text(const char *left, const char *right, const char *separator)
{
size_t left_length = strlen(left);
size_t right_length = strlen(right);
size_t separator_length = strlen(separator);
char *joined = malloc(left_length + separator_length + right_length + 1);
if (joined == NULL) {
perror("malloc");
exit(EXIT_FAILURE);
}
memcpy(joined, left, left_length);
memcpy(joined + left_length, separator, separator_length);
memcpy(joined + left_length + separator_length, right, right_length + 1);
return joined;
}
static char *copy_semantic_text(const char *text)
{
size_t length = strlen(text);
char *copy = malloc(length + 1);
if (copy == NULL) {
perror("malloc");
exit(EXIT_FAILURE);
}
memcpy(copy, text, length + 1);
return copy;
}
%}
Do you want to install the recommended 'C/C++ Extension Pack' extension from Microsoft for the C language?
Install
Show Recommendations
Outline
Timeline
Ln 1, Col 1 Spaces: 4 UTF-8 LF Plain Text Go Live Prettier


Code File Edit Selection View Go Run Terminal Window Help
Sun 11 Oct 1:37 PM
Compiler Construction
Explorer
Compiler Con...
.vscode
Assignment-1
Assignment-2
Assignment-3
Assignment-4
Assignment-5
Assignment-6
Assignment-7
Assignment-8
Assignment-9
.gitignore
grammar.h
lexer.l
lexer.yy.c
Makefile
parser.c
parser.tab.c
parser.tab.h
parser.y
rdparser
README.md
symbol_table.l
postfix.l
postfix.y
lexer.l Assignment-6
parser.y Assignment-6
parser.y Assignment-9
input.c
Assignment-9 > parser.y
%parse-param { Grammar *grammar }
%union {
char *text;
}
%token <text> SYMBOL
%token EPS ARROW PIPE NEWLINE
%type <text> alternative symbol_sequence alternatives
%%
production:
SYMBOL ARROW alternatives NEWLINE
{
int added = grammar_add_serialized_alternatives(grammar, $1, $3);
free($1);
free($3);
if (!added) {
YYABORT;
}
}
;
alternatives:
alternative
{
$$ = $1;
}
| alternatives PIPE alternative
{
$$ = join_text($1, $3, "|");
free($1);
free($3);
}
;
alternative:
Do you want to install the recommended 'C/C++ Extension Pack' extension from Microsoft for the C language?
Install
Show Recommendations
Outline
Timeline
Ln 1, Col 1 Spaces: 4 UTF-8 LF Plain Text Go Live Prettier


Code File Edit Selection View Go Run Terminal Window Help
Sun 11 Oct 1:37 PM
Compiler Construction
Explorer
Compiler Con...
.vscode
Assignment-1
Assignment-2
Assignment-3
Assignment-4
Assignment-5
Assignment-6
Assignment-7
Assignment-8
Assignment-9
.gitignore
grammar.h
lexer.l
lexer.yy.c
Makefile
parser.c
parser.tab.c
parser.tab.h
parser.y
rdparser
README.md
symbol_table.l
postfix.l
postfix.y
lexer.l Assignment-6
parser.y Assignment-6
parser.y Assignment-9
input.c
Assignment-9 > parser.y
alternative:
symbol_sequence
{
$$ = $1;
}
| EPS
{
$$ = copy_semantic_text("eps");
}
;
symbol_sequence:
SYMBOL
{
$$ = $1;
}
| symbol_sequence SYMBOL
{
$$ = join_text($1, $2, " ");
free($1);
free($2);
}
;
%%
void yyerror(Grammar *grammar, const char *message)
{
free(grammar->error_message);
grammar->error_message = NULL;
size_t length = 0;
while (message[length] != '\0') {
++length;
}
grammar->error_message = malloc(length + 1);
if (grammar->error_message != NULL) {
for (size_t i = 0; i <= length; ++i) {
Do you want to install the recommended 'C/C++ Extension Pack' extension from Microsoft for the C language?
Install
Show Recommendations
Outline
Timeline
Ln 1, Col 1
Spaces: 4
UTF-8
LF
Plain Text
Go Live
Prettier


Code File Edit Selection View Go Run Terminal Window Help
Sun 11 Oct 1:37 PM
Compiler Construction
Explorer
Compiler Con...
.vscode
Assignment-1
Assignment-2
Assignment-3
Assignment-4
Assignment-5
Assignment-6
Assignment-7
Assignment-8
Assignment-9
.gitignore
grammar.h
lexer.l
lexer.yy.c
Makefile
parser.c
parser.tab.c
parser.tab.h
parser.y
rdparser
README.md
symbol_table.l
postfix.l
postfix.y
lexer.l Assignment-6
parser.y Assignment-6
parser.y Assignment-9
input.c
Assignment-9 > parser.y
alternative:
| EPS
{
}
;
symbol_sequence:
SYMBOL
{
$$ = $1;
}
| symbol_sequence SYMBOL
{
$$ = join_text($1, $2, " ");
free($1);
free($2);
}
;
%%
void yyerror(Grammar *grammar, const char *message)
{
free(grammar->error_message);
grammar->error_message = NULL;
size_t length = 0;
while (message[length] != '\0') {
++length;
}
grammar->error_message = malloc(length + 1);
if (grammar->error_message != NULL) {
for (size_t i = 0; i <= length; ++i) {
grammar->error_message[i] = message[i];
}
}
}
Do you want to install the recommended 'C/C++ Extension Pack' extension from Microsoft for the C language?
Install
Show Recommendations
Outline
Timeline
Ln 1, Col 1
Spaces: 4
UTF-8
LF
Plain Text
Go Live
Prettier